OAP200V

Object-oriented analysis, design and programming

02 – Object-oriented design principles

Lecturer: Boban Vesin

Outline

- Design principles are generalized pieces of advice or proven good coding practices that are used as rules of thumb when making design choices.
- They're a similar concept to design patterns, the main difference being that design principles are more abstract and generalized. They are high-level pieces of advice, often applicable to many different programming languages or even different paradigms
- Design patterns are also abstractions or generalized good practices, but they provide much more concrete and practical low-level advice, and are related to entire classes of problems rather than just generalized coding practices.

OO principles

Inheritance

- **Description**: Inheritance is a mechanism where a new class, known as a subclass or derived class, is created based on an existing class, called the superclass. The subclass inherits fields and methods from the superclass.
- **Purpose**: Inheritance promotes code reuse and establishes a relationship between the superclass and subclass.
- **How It Looks in Java**:
  - The extends keyword is used to create a subclass.
  - The subclass inherits fields and methods from the superclass and can override them if needed.

Inheritance

- One object acquires all the properties and behaviours of a parent object
- Parent-child relationship between two classes
- Benefits:
  - Implements the DRY (don´t repeat yourself) programming principle
  - Improves code reusability

```
class Animal {
    void eat() {
        System.out.println("I am a omnivorous!! ");
    }
}

class Mammal extends Animal {
    void nature() {
        System.out.println("I am a mammal!! ");
    }
}

class Dog extends Mammal {
    void sound() {
        System.out.println("I bark!! ");
    }
}

class Inheritance {
    public static void main(String args[]) {
        Dog d = new Dog();
        d.eat();
        d.nature();
        d.sound();
    }
}
```

Abstraction

- **Description**: Abstraction is the concept of hiding complex implementation details and showing only the necessary features of an object. In Java, this is achieved using abstract classes and interfaces.
- **Purpose**: Abstraction reduces complexity and allows the developer to focus on interactions at a high level.
- **How It Looks in Java**:
  - **Abstract Classes**: Declared with the abstract keyword, and cannot be instantiated. They may contain abstract methods (without implementations) and concrete methods (with implementations).
  - **Interfaces**: Define a contract for classes to implement, specifying only method signatures but no implementations.

Abstraction

- Act of reducing programming complexity by representing essential features without including the background explanations or details
- Benefits:
  - Hides the underlying complexity of data
  - Helps avoid repetitive code

```
abstract class Car {
    Car() {
        System.out.println("Car is built. ");
    }

    abstract void drive();

    void gearChange() {
        System.out.println("Gearchanged!!");
    }
}

class Tesla extends Car {
    void drive() {
        System.out.println("Drive Safely");
    }
}

class Abstraction {
    public static void main(String args[]) {
        Car obj = new Tesla();
        obj.drive();
        obj.gearChange();
    }
}
```

Polymorphism

- **Description**: Polymorphism allows objects of different classes to be treated as objects of a common superclass. In Java, this is achieved through method overriding and interfaces.
- **Purpose**: Polymorphism enables flexibility and allows for code that can operate on objects of different types without knowing their exact class.
- Two forms:
  - method overloading
  - method overriding

Method overloading

- methods with the same name but different parameter lists and different definitions

```
class CircleArea {
    double area(double x) {
        return 3.14 * x;
    }
}

class SquareArea {
    int area(int x) {
        return x * x;
    }
}

class RectangleArea{
    int area(int x, int y) {
        return x * y;
    }
}

class TriangleArea{
    int area(int y, int x) {
        return (y * x) / 2;
    }
}

class Overloading{
    public static void main(String args[]) {
        CircleArea ca = new CircleArea();
        SquareArea sa = new SquareArea();
        RectangleArea ra = new RectangleArea();
        TriangleArea ta = new TriangleArea();

        System.out.println("Circle area = " + ca.area(1));
        System.out.println("Square area = " + sa.area(2));
        System.out.println("Rectangle area = " + ra.area(3, 4));
        System.out.println("Triangle area = " + ta.area(6, 3));
    }
}
```

Method overriding

- A method defined in the superclass is inherited by its subclass and is used by the objects created by the subclass
- However, there may be occasions when an object should respond to the same method but behave differently when that method is called, which means a method defined in the superclass is overridden.

```
class Shape {
    void draw() {
        System.out.println("Mention shape here");
    }

    void numberOfSides() {
        System.out.println("side = 0");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("CIRCLE ");
    }

    @Override
    void numberOfSides() {
        System.out.println("side = 0 ");
    }
}
```

Encapsulation

- **Description**: Encapsulation is the bundling of data (fields) and methods that operate on that data within a single unit, typically a class. It also involves restricting direct access to some of an object's components, which is often achieved by using access modifiers (private, protected, public).
- **Purpose**: This principle ensures that the internal representation of an object is hidden from the outside, protecting it from unintended interference and misuse.
- **How It Looks in Java**:
  - Fields are often marked private.
  - Public getter and setter methods control access to these private fields.

Encapsulation

- The wrapping up of the data and methods into the single unit is known as encapsulation
- Making the fields (class variables) private and accessing them via their public getter and setter methods
- Benefits:
  - Provides information hiding
  - Restricts direct access to data

```
class EmployeeCount {
    private int numOfEmployees = 0;

    public void setNoOfEmployees(int count) {
        numOfEmployees = count;
    }

    public double getNoOfEmployees() {
        return numOfEmployees;
    }
}
```

Design principles

Keep It Simple and Stupid (KISS) Principle

- The Keep it Simple and Stupid (KISS) principle is a reminder to keep your code simple and readable for humans
  - If your method handles multiple use-cases, split them into smaller functions.
  - If it performs multiple functionalities, make multiple methods instead.
- The core of this principle is that for most cases, unless efficiency is extremely crucial, another stack call isn't going to severely affect the performance of your program. In fact, some compilers or runtime environments will even simplify a method call into an inline execution.

DRY - Don’t repeat yourself

- Every piece of knowledge or logic must have a single, unambiguous representation within a system
- Don’t write duplicate code
  - instead use Abstraction to abstract everyday things in one place
- If you have a block of code in more than two places, consider making it a separate method, or if you use a hard-coded value more than one time, make them public final constant
- this means utilizing abstract classes, interfaces, and public constants

DRY - Don’t repeat yourself - example

- Both a Cat and a Dog need to eat food, but they speak differently. Since eating food is a common functionality for them, we can abstract it into a parent class such as Animal and then have them extend the class.
- Now, instead of both classes implementing the same functionality of eating food, each can focus on their own unique logic

```
public class Animal {
    public void eatFood() {
        System.out.println("Eating food...");
    }
}

public class Cat extends Animal {
    public void meow() {
        System.out.println("Meow! *purrs*");
    }
}

public class Dog extends Animal {
    public void woof() {
        System.out.println("Woof! *wags tail*");
    }
}
```

Encapsulate what changes

- There is only one thing which is constant in the software field, and that is “Change,”
  - Encapsulate the code you expect or suspect to be changed in the future.
- Follow the principle of making variables and methods private by default and increasing access step-by-step

The interface segregation principle (ISP)

- The client should never be forced to depend on an interface they aren't using in its entirety
- This means that an interface should have a minimum set of methods necessary for the functionality it ensures, and should be limited to only one functionality
- For example, a Pizza interface shouldn't be required to implement an addPepperoni() method, because this doesn't have to be available for every type of pizza

The interface segregation principle (ISP) - example

- If the addMushrooms() or addPepperoni() methods were located in the interface, both classes would have to implement them even though they don't need both, but rather only one each.

```
public interface Pizza {
    void addSauce();
    void bake();
}
```

```
public class VegetarianPizza implements Pizza {
    public void addMushrooms() {System.out.println("Adding mushrooms");}

    @Override
    public void addSauce() {System.out.println("Adding sauce");}

    @Override
    public void bake() {System.out.println("Baking the vegetarian pizza");}
}

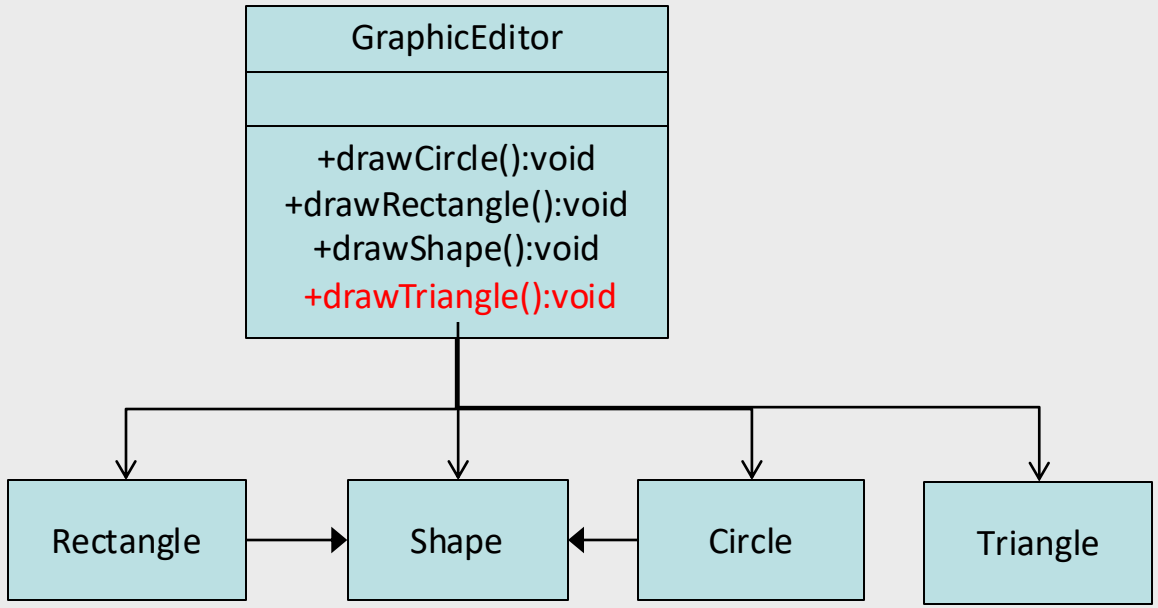
public class PepperoniPizza implements Pizza {
    public void addPepperoni() {System.out.println("Adding pepperoni");}

    @Override
    public void addSauce() {System.out.println("Adding sauce");}

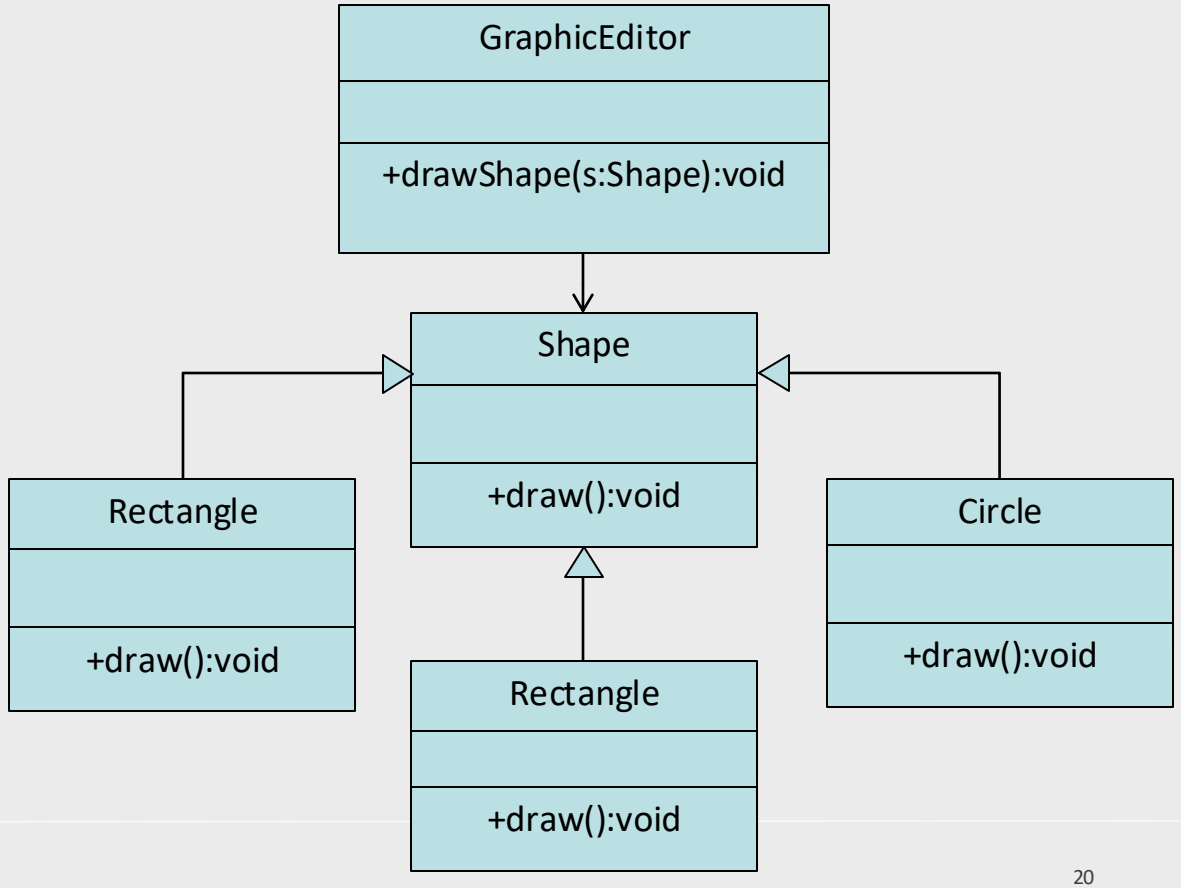
    @Override
    public void bake() {System.out.println("Baking the pepperoni pizza");}
}
```

Open/closed design principle

- Classes, methods, or functions should be open for extension (new functionality) and closed for modification



When a new shape is added this should be changed



No changes required when a new shape is added.

Single responsibility principle - SRP

- There should never be two functionalities in one class
- A class should only have one, and only one, reason to be changed (responsibility of the class)
- For example, If you put more than one functionality in one Class in Java, it introduces coupling between two functionalities, and even if you change one feature, there is a chance you broke coupled functionality, which requires another round of testing to avoid any surprise on the production environment.

Programming styles

Coupling

- Coupling refers to the extent to which a class knows about the other class
  - How related or dependent two classes/modules are toward each other
- There are two types of coupling
  - High (tight) Coupling(a bad programming design)
  - Low (loose) Coupling (a good programming design)
- For low coupled classes, changing something major in one class should not affect the other
- High coupling would make it difficult to change and maintain your code; since classes are closely knit together, making a change could require an entire system revamp

High coupling example

High coupling: Directly accessing the value of data member "name" of class A, due to high coupling between the two classes

```
classA{
	public String name; //public data member of A class
	public String getName(){
		//Checking a valid access to "name"
		if(name!=null)
			return name;
		else
			return "not initiaized";
	}

	public void setName(String s){
	//Checking a valid setting to "name"
		if (s==null){
			System.out.println("You can't initialized name to a null");
		}
	}

}
```

```
classB{
	public static void main(String[] args){
		A ob= new A();

		ob.name=null;
		System.out.println("Name is " + ob.name);
	}
}
```

Low coupling example

Low coupling: Calling setter method, as the direct access of "name" is not possible

```
classA{
	private String name; //public data member of A class
	public String getName(){
		//Checking a valid access to "name"
		if(name!=null)
			return name;
		else
			return "not initiaized";
	}

	public void setName(String s){
	//Checking a valid setting to "name"
		if (s==null){
			System.out.println("You can't initialized name to a null");
		}
	}

}
```

```
classB{
	public static void main(String[] args){
		A ob= new A();

		ob.setName(null);
		System.out.println("Name is " + ob.getName());

	}
}
```

Cohesion

- Cohesion refers to the extent to which a class is defined to do a specific specialized task. A class created with high cohesion is targeted towards a single specific purpose, rather than performing many different specific purposes.
- There are two types of cohesion
  - Low cohesion(a bad programming design)
  - High Cohesion(a good programming design)

Low cohesion example

- When a class is designed to do many different tasks rather than focus on a single specialized task

```
class PlayerDatabase{
     public void connectDatabase();
     public void printAllPlayersInfo();
     public void printSinglePlayerInfo();
     public void printRankings()
     public void closeDatabase();
}
```

- such a class is not easy to create, maintain and update, as it is involved in doing so many different tasks i.e. a programming design to avoid.

High cohesion example

- A good application design is creating an application with high cohesive classes, which are targeted towards a specific specialized task and such classes are not only easy to create but also easy to maintain and update.

```
class DatabaseConnection{//managing connection to database.
      public void connectDatabase();
      public void closeDatabase();
}

class PrintData{ //management of printing
      public void printAllPlayersInfo();
      public void printSinglePlayerInfo();
      public void printRankings()
}
```

Questions?